CP Generator: Formal Notes

Allan Pan

May 24, 2026

1 The implemented mathematical object

The project works with a square sheet

$$S = [0, s]^2, \qquad s > 0,$$

and with a finite straight-line graph drawn on that sheet. More precisely, the implemented state is a triple

$$\mathcal{C} = (G, X, \tau),$$

where $G = (V, E)$ is a finite graph, $X \colon V \to S$ is an embedding, and

$$\tau \colon E \to \{-1, 0, 1\}$$

is a fold-label map, with -1 for "unassigned", 0 for "mountain", and 1 for "valley".

Definition 1.1 (Boundary and interior vertices)**.**

$$V_\partial = \{v \in V : X_v \in \partial S\}, \qquad V_{\text{int}} = V \setminus V_\partial.$$

Every local theorem used by the program is expressed at interior vertices, so the distinction between V_∂ and V_{int} is structural rather than cosmetic.

Definition 1.2 (Local angle data)**.** *For $v \in V_{\text{int}}$, let*

$$d(v) = \deg_G(v).$$

If the incident creases at v are listed in cyclic order, then the corresponding sector angles are written

$$\alpha_1(v), \dots, \alpha_{2m(v)}(v) \in (0, 2\pi), \qquad \sum_{i=1}^{2m(v)} \alpha_i(v) = 2\pi.$$

Definition 1.3 (Kawasaki residual and Maekawa balance)**.** *For $v \in V_{\text{int}}$, define*

$$\kappa(v) = \max\left(\left|\sum\nolimits_{\text{i odd}} \alpha_i(v) - \pi\right|, \left|\sum\nolimits_{\text{i even}} \alpha_i(v) - \pi\right|\right).$$

If τ is complete at v, define

$$M_\tau(v) = \#\{e \ni v : \tau(e) = 0\}, \qquad V_\tau(v) = \#\{e \ni v : \tau(e) = 1\}, \qquad \mu_\tau(v) = M_\tau(v) - V_\tau(v).$$

2 From random points to a planar candidate graph

The first stage of the project is purely geometric. One samples interior points

$$P \subset (0, s)^2,$$

adjoins the four corners of the square, and then projects points lying within a fixed threshold $\delta > 0$ of the boundary onto ∂S. Thus one obtains

$$P^{\square} = P \cup \{(0,0), (s,0), (s,s), (0,s)\}, \qquad \widetilde{P} = \Pi_{\partial,\delta}(P^{\square}).$$

The candidate graph is then taken from the Delaunay triangulation of $\widetilde{P}$:

$$G_0 = \mathrm{Del}(\widetilde{P}).$$

After this, the code removes every edge lying entirely on one side of the square boundary. Therefore the implemented crease graph is

$$E = E(G_0) \setminus E_{\partial},$$

where

$$E_{\partial} = \{\{u, v\} \in E(G_0) : X_u, X_v \text{ lie on one boundary side of } S\}.$$

This Delaunay stage is not itself a flat-foldability theorem. Its role is to provide a planar graph with reasonably regular local geometry so that the later angle constraints are numerically meaningful.

3 Workflow-level genericity guard

The local flat-foldability equations alone do not prevent a numerically collapsed sample in which two vertices become arbitrarily close. The workflow therefore adds a scale-normalized geometric genericity test before it accepts a sample as "green".

Definition 3.1 (Minimum vertex separation)**.** *For an embedded pattern $\mathcal{C} = (G, X, \tau)$ on a square of side length s, define*

$$\delta_{\min}(X) = \min_{\substack{u,v \in V \\ u \neq v}} \|X_u - X_v\|, \qquad \rho(X) = \frac{\delta_{\min}(X)}{s}.$$

Definition 3.2 (Implemented generic-geometry predicate)**.** *The workflow fixes the ratio threshold*

$$\rho_{\min} = 10^{-3}.$$

It calls the geometry generic *when*

$$\delta_{\min}(X) \geq \rho_{\min} s.$$

Equivalently,

$$\mathrm{Generic}(X) \iff \rho(X) \geq 10^{-3}.$$

The pattern generator retries up to a fixed maximum number of random samples and returns the first generic one. If no generic sample is found, it keeps the best available sample, namely the one maximizing $\delta_{\min}(X)$ among those attempts. Likewise, the local optimizer is declared "green" only when

$$\mathrm{Local}(\mathcal{C}; \varepsilon) \quad \text{and} \quad \mathrm{Generic}(X)$$

hold simultaneously.

Definition 3.3 (Workflow-level local green)**.** *The workflow-level local success predicate is*

$$\mathrm{LocalGreen}(\mathcal{C};\varepsilon) \Longleftrightarrow \mathrm{Local}(\mathcal{C};\varepsilon) \wedge \mathrm{Generic}(X).$$

Remark 3.4. This genericity guard is not a theorem from classical origami theory. It is an implemented numerical nondegeneracy assumption, used to reject nearly coincident vertices for which angle diagnostics and later exact-face reasoning become unstable.

4 The local flat-foldability conditions

The next stage turns the geometric candidate graph into a constrained origami object by imposing the standard single-vertex necessary conditions.

Theorem 4.1 (Single-vertex necessary conditions [2])**.** *If $v \in V_{\mathrm{int}}$ is flat-foldable as a single-vertex crease pattern, then*

$$d(v) \in 2\mathbb{Z}, \qquad \sum\nolimits_{\mathrm{i\ odd}} \alpha_i(v) = \pi, \qquad \sum\nolimits_{\mathrm{i\ even}} \alpha_i(v) = \pi, \qquad |\mu_\tau(v)| = 2.$$

The optimizer in the code imposes only one alternating-angle equality per vertex, not two. This is sufficient, because the second equality is redundant.

Lemma 4.2 (Alternating-sum redundancy)**.** *If*

$$\sum_{i=1}^{2m} \alpha_i = 2\pi,$$

then

$$\sum\nolimits_{\mathrm{i\ odd}} \alpha_i = \pi \Longleftrightarrow \sum\nolimits_{\mathrm{i\ even}} \alpha_i = \pi.$$

Proof. Since

$$\sum\nolimits_{\mathrm{i\ even}} \alpha_i = 2\pi - \sum\nolimits_{\mathrm{i\ odd}} \alpha_i,$$

each equality implies the other. □

Accordingly, the project defines its local diagnostic predicate by

$$\mathrm{Local}(\mathcal{C};\varepsilon) \Longleftrightarrow \forall v \in V_{\mathrm{int}} \ \big(d(v) \in 2\mathbb{Z} \ \wedge \ \kappa(v) \le \varepsilon\big).$$

This is the mathematical content of the program's *local status*.

5 Parity repair before optimization

The first obstruction to local flat-foldability is odd degree. Let

$$O(G) = \{v \in V : d(v) \notin 2\mathbb{Z}\}$$

denote the odd-degree set. The implementation repairs parity by repeatedly deleting shortest paths between odd vertices.

Definition 5.1 (Implemented parity-repair step)**.** *Choose*

$$u \in O(G), \qquad w = \operatorname*{arg\,min}_{z \in O(G)\setminus\{u\}} d_G(u, z),$$

and let

$$\gamma = (u = v_0, v_1, \ldots, v_k = w)$$

be a shortest path from u to w in G. Replace E by

$$E \setminus \{\{v_{i-1}, v_i\} : 1 \le i \le k\}.$$

The reason this step is mathematically natural is that path deletion affects parity only at the endpoints.

Proposition 5.2 (Parity effect of one deletion)**.** *If the path γ above is deleted, then*

$$\deg(v_i) \mapsto \deg(v_i) - 2 \qquad (1 \le i \le k-1),$$

and

$$\deg(u) \mapsto \deg(u) - 1, \qquad \deg(w) \mapsto \deg(w) - 1.$$

Hence the parity of every internal path vertex is preserved, while the parity of the endpoints flips.

Proof. Each internal path vertex loses exactly two incident edges, and each endpoint loses exactly one. □

This proves that the repair has the intended local parity effect. It does not prove that the repaired graph is globally optimal in any stronger sense.

6 Weighted geometric repair via constrained optimization

Once parity has been repaired, the code moves the vertices as little as possible while imposing Kawasaki's identity at every interior vertex.

Let $X^{(0)} = (X_v^{(0)})_{v \in V}$ denote the pre-optimization coordinates. Boundary motion is heavily penalized by weights

$$w_v = \begin{cases} 10^6, & v \in V_\partial, \\ 1, & v \in V_{\text{int}}. \end{cases}$$

The weighted displacement functional is

$$D(X) = \sum_{v \in V} w_v \left\| X_v - X_v^{(0)} \right\|^2.$$

The actual code minimizes

$$F(X) = \frac{0.1}{2} D(X)^2.$$

Since $D(X) \ge 0$, one has

$$\operatorname{arg\,min} F = \operatorname{arg\,min} D.$$

Thus the implementation still solves the nearest-feasible-geometry problem in the weighted least-squares sense.

For each interior vertex v, define the constraint

$$g_v(X) = \sum\nolimits_{\text{i odd}} \alpha_i(v; X) - \pi.$$

The implemented optimization problem is therefore

$$\min_X F(X) \qquad \text{subject to} \qquad g_v(X) = 0 \quad \forall v \in V_{\text{int}}. \tag{1}$$

The solver is SLSQP. After the numerical solve, vertices within a small tolerance of the boundary are snapped back onto ∂S. In this way, the project passes from a merely planar sample to a planar sample constrained by the local angle theorem.

7 Mountain–valley search by Bern–Hayes-style local reduction

After geometric repair, the remaining local problem is to assign mountain and valley labels. The code does not search over all edge labelings directly. Instead, it first extracts local sign relations from smallest sectors, following the Bern–Hayes reduction idea [1].

For an interior vertex v, let

$$e_1, \ldots, e_r$$

be the incident creases in cyclic order, and let

$$\beta_1, \ldots, \beta_r$$

be the corresponding sector angles. The reduction repeatedly removes a locally minimal sector and records whether its two boundary creases must carry the same or opposite mountain–valley sign.

Definition 7.1 (Local reduction chain)**.** *The reduction at v produces*

$$\Gamma_v = \big((f_1, \ldots, f_k), (\eta_1, \ldots, \eta_k)\big), \qquad \eta_i \in \{0, 1, 2\},$$

where

$$\eta_i = \begin{cases} 0, & \tau(f_i) = 1 - \tau(f_{i-1}), \\ 1, & \tau(f_i) = \tau(f_{i-1}), \\ 2, & \tau(f_i) \text{ remains undetermined.} \end{cases}$$

Boundary vertices are treated similarly, except that temporary virtual boundary rays are added so that the cyclic order closes. Thus the local reduction stage produces one or more chains of sign relations at every vertex.

8 Exact enumeration after chain merging

The local chains are then merged across shared creases. If the merged chains are

$$\Lambda_1, \ldots, \Lambda_g,$$

then each Λ_j contributes one free binary choice, namely the sign of its first crease. Therefore the remaining search space is

$$\{0, 1\}^g.$$

For

$$c = (c_1, \ldots, c_g) \in \{0,1\}^g,$$

the code propagates the implied signs along each chain:

$$\tau_c(e_{j,1}) = c_j,$$

and then

$$\eta_{j,i} = 0 \Longrightarrow \tau_c(e_{j,i}) = 1 - \tau_c(e_{j,i-1}),$$
$$\eta_{j,i} = 1 \Longrightarrow \tau_c(e_{j,i}) = \tau_c(e_{j,i-1}),$$
$$\eta_{j,i} = 2 \Longrightarrow \tau_c(e_{j,i}) = -1.$$

This yields a partial or complete labeling τ_c. The exact admissibility filter is Maekawa's theorem:

$$\mathrm{Adm}(c) \Longleftrightarrow \forall v \in V_{\mathrm{int}} \ \ |\mu_{\tau_c}(v)| = 2.$$

Among admissible choices, the code chooses the one maximizing an interlacing score

$$\mathrm{Score}(c) = 0.05\,\#\{e \in E : \tau_c(e) \in \{0,1\}\} \sum_{v \in V} \Phi_v(c) - 4\,G(\tau_c),$$

where $\Phi_v(c)$ rewards alternating mountain–valley patterns more than repeating ones, with sharper wedges weighted more heavily, and where $G(\tau_c)$ counts the obtuse-angle defects defined in the next section. This score is not a correctness condition; it is only a tie-breaker among already admissible assignments.

Proposition 8.1 (What the assignment search certifies)**.** *If the routine returns an assignment τ^*, then*

$$\forall v \in V_{\mathrm{int}} \ \ |\mu_{\tau^*}(v)| = 2.$$

If the routine returns failure, then no enumerated merged-group assignment satisfies Maekawa's theorem at every interior vertex.

Proof. The code checks every seed choice in $\{0,1\}^g$, rejects every choice violating Maekawa, and returns only from the surviving set. □

9 Maekawa-preserving repair of obtuse monochrome glitches

Local Maekawa admissibility is still not enough to rule out every visibly bad mountain–valley pattern. The implementation now singles out one specific local configuration that repeatedly produced globally implausible flat stacks.

Definition 9.1 (Obtuse monochrome glitch)**.** *Let $v \in V_{\mathrm{int}}$, and let*

$$e_i, e_{i+1}, e_{i+2}$$

be three consecutive incident creases in cyclic order, with consecutive sector angles

$$\beta_i, \beta_{i+1}.$$

For a complete local assignment τ, this triple is called an obtuse monochrome glitch *if*

$$\beta_i > \frac{\pi}{2}, \qquad \beta_{i+1} > \frac{\pi}{2}, \qquad \tau(e_i) = \tau(e_{i+2}) \neq \tau(e_{i+1}).$$

Equivalently, two consecutive obtuse wedges are bounded by a "same–different–same" sign pattern. In the user-facing language of the program, the three creases should instead become monochromatic.

Definition 9.2 (Two monochromizing moves)**.** *For a glitch triple* (e_i, e_{i+1}, e_{i+2})*, the code tests exactly two local updates:*

$$\textit{(middle flip)} \qquad \tau'(e_{i+1}) = \tau(e_i),$$

and

$$\textit{(outer flip)} \qquad \tau''(e_i) = \tau''(e_{i+2}) = \tau(e_{i+1}),$$

with every other crease unchanged.

Only one of these moves is accepted, and only when it preserves Maekawa's theorem not merely at v, but at every interior vertex.

Definition 9.3 (Accepted glitch repair)**.** *For a glitch triple* (e_i, e_{i+1}, e_{i+2}) *at* v*, an update*

$$\tau \mapsto \widetilde{\tau}$$

is accepted *if it is one of the two monochromizing moves above and*

$$|\mu_{\widetilde{\tau}}(w)| = 2 \qquad \forall w \in V_{\text{int}}.$$

The repair loop then chooses among all accepted updates by the lexicographic rule

$$\big(G(\widetilde{\tau}),\ \#\text{changed creases},\ \operatorname{Sig}(\widetilde{\tau})\big),$$

where smaller glitch count wins first, then fewer changed creases, then the assignment signature used by the code for deterministic tie-breaking. The loop repeats until no accepted update decreases G or a previously seen signature would be revisited.

Proposition 9.4 (What the repair loop guarantees)**.** *If the obtuse-glitch repair routine terminates with assignment* $\widehat{\tau}$*, then every accepted intermediate update preserved Maekawa's theorem at all interior vertices, and the final assignment has no smaller reachable glitch count among the one-step accepted updates considered from the final state.*

Proof. Every candidate update is explicitly checked against Maekawa's theorem on every interior vertex before acceptance. The loop only commits an update when it strictly improves the current glitch count, and it halts once no such improving accepted update exists or a repeated signature would occur. □

10 From local certificates to global geometry

At this point, the project has local angle control and a locally admissible mountain–valley labeling. The next question is global: whether the current straight-line embedding can be organized into coherent folded faces.

The first global obstruction is crossing.

Definition 10.1 (Crossing predicate)**.** *For distinct nonincident edges* $e, e' \in E$*, define*

$$\operatorname{Cross}(e, e'; X) \iff X(e) \cap X(e') \neq \varnothing.$$

Set

$$\operatorname{Noncross}(X) \iff \forall e \neq e' \in E, \quad e \cap e' = \varnothing \implies \neg \operatorname{Cross}(e, e'; X).$$

If $\operatorname{Noncross}(X)$ fails, then no exact face-based folded preview is attempted on the current geometry.

11 Exact face extraction by half-edge tracing

Assume now that the current embedding is noncrossing. The code augments the fold graph by the four boundary edges of the square:

$$E^{\square} = E \cup E_{\partial S}.$$

At each vertex v, neighbors are ordered by polar angle around X_v. This induces a left-face successor rule on oriented edges.

Definition 11.1 (Left-face successor)**.** *For an oriented edge (u, v), let*

$$\sigma(u, v) = (v, w),$$

where w is the predecessor of u in the cyclic order around v.

Starting from any half-edge and iterating σ yields a closed walk or a failure. Every bounded counterclockwise closed walk is declared to be a face. Repeated vertices or nonclosing walks cause the exact solver to abort.

Thus the code produces a finite face family

$$\mathcal{F}.$$

For each fold edge $e \in E$, one then computes its face-incidence set

$$\mathrm{Inc}(e) = \{f \in \mathcal{F} : e \subset \partial f\}.$$

The exact solver requires

$$|\mathrm{Inc}(e)| = 2 \qquad \forall e \in E.$$

This is the precise global topological condition needed before reflection propagation can begin.

12 Reflection propagation and exact-current-geometry certification

Suppose every fold edge borders exactly two faces. The code chooses a root face

$$f_0 \in \mathcal{F}, \qquad \mathrm{area}(f_0) = \max_{f \in \mathcal{F}} \mathrm{area}(f),$$

and sets

$$T_{f_0} = I_3.$$

If faces f and g share a fold line ℓ, then the child transform is defined by reflection:

$$T_g = R_\ell T_f.$$

When a face is reached by two different reflection paths, the resulting transforms are compared on the vertices of that face.

Definition 12.1 (Cycle discrepancy)**.** *For a face f and two candidate transforms T, T', define*

$$\Delta(f; T, T') = \max_{v \in \mathrm{Vert}(f)} \left\| T(X_v) - T'(X_v) \right\|.$$

If

$$\Delta(f; T, T') > \Delta_{\text{hard}},$$

then exact reconstruction fails. If

$$\Delta_{\text{soft}} < \Delta(f; T, T') \le \Delta_{\text{hard}},$$

then the reconstruction survives only at warning level.

Fold signs are resolved from τ by

$$\tau(e) = 0 \Longrightarrow \operatorname{sgn}(e) = +1, \qquad \tau(e) = 1 \Longrightarrow \operatorname{sgn}(e) = -1.$$

If a fold remains unassigned, the exact solver fills its sign provisionally by a face-depth rule; this also downgrades the output to warning level.

Theorem 12.2 (Exact-current-geometry certificate)**.** *If the project reports*

$$\text{Status}_{\text{global}}(\mathcal{C}) = \text{Pass} \qquad \textit{and} \qquad \text{Status}_{\text{preview}}(\mathcal{C}) = \text{Pass},$$

then

$$\text{Noncross}(X), \qquad |\text{Inc}(e)| = 2 \ \forall e \in E, \qquad \Delta(f; T, T') \le \Delta_{\text{soft}}$$

for every cycle comparison, and no provisional fold signs were used.

Proof. Each clause is the negation of one of the exact solver's failure or warning branches. A pass can occur only if every one of those branches is avoided. □

This theorem describes the strongest global statement currently certified by the code. It is a theorem about the *current embedding*, not a complete classification theorem for all embeddings with the same combinatorics.

13 Bounded exact stack-order checking

The exact preview solver reconstructs faces and rigid transforms, but it does not by itself enumerate all possible flat overlap orders. For sufficiently small exact instances, the project therefore runs a separate bounded exact checker on the extracted face arrangement.

Definition 13.1 (Bounded-check prerequisites)**.** *The bounded checker runs only if all of the following hold:*

1. $\text{Status}_{\text{local}}(\mathcal{C}) = \text{Pass}$,
2. $\text{Status}_{\text{assign}}(\mathcal{C}) = \text{Pass}$,
3. $\text{Noncross}(X)$,
4. *exact face reconstruction succeeds,*
5. *no provisional fold signs are used,*
6. *no approximate cycle closure is used, and*

7. *the extracted face count satisfies*

$$|\mathcal{F}| \le F_{\max}, \qquad F_{\max} = 8$$

in the current implementation.

The checker compares faces in a near-flat, but still slightly separated, rigid pose. The implemented angle is

$$\theta_{\mathrm{nf}} = \pi - 0.08.$$

Let

$$P_f^{\mathrm{nf}} \subset \mathbb{R}^3$$

be the points of face f at angle θ_{nf}, and let

$$\overline{P}_f \subset \mathbb{R}^2$$

be the corresponding exact final flat projection.

Definition 13.2 (Overlap cell)**.** *Let $T \subset \overline{P}_f$ and $U \subset \overline{P}_g$ be triangles from the face triangulations. Their* overlap cell *is the convex polygon*

$$C(T,U) = T \cap U$$

computed by polygon clipping. Cells of area at most

$$\varepsilon_{\mathrm{area}} = 10^{-8}$$

are discarded.

For each nontrivial overlap cell, the code samples the centroid

$$c(T,U) \in C(T,U)$$

and interpolates the two near-flat heights

$$z_f\big(c(T,U)\big), \qquad z_g\big(c(T,U)\big)$$

from the corresponding 3-dimensional triangles by barycentric coordinates.

Definition 13.3 (Pairwise overlap-order relation)**.** *For two distinct faces f, g, if every overlap cell between them satisfies*

$$z_f(c) - z_g(c) > \varepsilon_z, \qquad \varepsilon_z = 10^{-7},$$

then the checker records the relation

$$f \succ g.$$

If every overlap cell instead satisfies

$$z_g(c) - z_f(c) > \varepsilon_z,$$

it records

$$g \succ f.$$

If some overlap cells reverse the sign, or if

$$|z_f(c) - z_g(c)| \le \varepsilon_z$$

on a nontrivial overlap, the checker fails immediately.

Thus the checker constructs a directed relation

$$R \subset \mathcal{F} \times \mathcal{F},$$

interpreted as a partial overlap order on faces. The bounded exact problem is then whether R admits a linear extension.

Definition 13.4 (Bounded stack order)**.** *A* bounded stack order *for the extracted arrangement is a permutation*

$$(f_{i_1}, \ldots, f_{i_m})$$

of $\mathcal{F}$ *such that*

$$(f, g) \in R \Longrightarrow f \text{ appears before } g.$$

The implementation counts bounded stack orders by a bitmask dynamic program. If

$$\mathrm{Pred}(f) = \{g : (g, f) \in R\},$$

and if $N(S)$ denotes the number of admissible completions from the already placed face set $S \subseteq \mathcal{F}$, then

$$N(S) = \sum_{\substack{f \in \mathcal{F} \setminus S \\ \mathrm{Pred}(f) \subseteq S}} N(S \cup \{f\}), \qquad N(\mathcal{F}) = 1.$$

The count is truncated above by

$$N_{\max} = 100000.$$

Proposition 13.5 (Meaning of a bounded exact-check pass)**.** *If the bounded checker returns* Pass*, then the extracted relation R admits at least one bounded stack order. If it returns* Pass *with unique-order flag, then exactly one bounded stack order was found before the truncation limit was reached.*

Proof. The dynamic program above counts precisely the linear extensions of the predecessor relation encoded by R, truncated only when the count exceeds $N_{\max}$. A positive count therefore certifies existence of at least one linear extension, and count 1 certifies uniqueness. □

Proposition 13.6 (Meaning of a cyclic bounded failure)**.** *If the bounded checker returns failure with message "cyclic stack-order constraint", then the extracted relation R has no linear extension, hence no bounded stack order compatible with all sampled overlap constraints.*

Proof. The cited failure branch is reached exactly when the topological-order counter returns 0. A directed acyclic predecessor relation has at least one linear extension, so zero count means that the extracted overlap-order constraints are incompatible. □

Remark 13.7. This bounded checker is still not a complete global flat-foldability algorithm. It is a small-instance certificate on the exact face arrangement actually constructed by the solver, and it is philosophically closest to the overlap-order viewpoint used in the flat-folding complexity literature [1, 5].

14 3D folding and fallback preview

Once exact face reconstruction is available, the project builds a kinematic 3D folding by propagating rotations along the face-adjacency tree. If a crease e has axis (p, q) and sign $\operatorname{sgn}(e) \in \{\pm 1\}$, then the 3-dimensional transform satisfies

$$T_g^{(3D)} = R_{(p,q)}\big(\operatorname{sgn}(e)\theta\big) T_f^{(3D)}.$$

This rigid-hinge viewpoint is the standard rigid-plate abstraction used for the preview stage; compare the rigid-origami literature [3, 4].

For time parameter $t \in [0, 1]$, the implementation uses the easing profile

$$\eta(t) = \frac{1 - \cos(\pi t)}{2},$$

and for any settle window $[a, b] \subseteq [0, 1]$ it uses the smoothstep blend

$$s_{a,b}(t) = \begin{cases} 0, & t \le a, \\ 3u(t)^2 - 2u(t)^3, & a \le t \le b, \quad u(t) = \dfrac{t-a}{b-a}, \\ 1, & t \ge b. \end{cases}$$

Let $\Pi : \mathbb{R}^3 \to \mathbb{R}^2$ be projection to the (x, y)-plane, let X_f denote the ordered source vertices of face f with initial z-coordinate 0, and define the two exact rigid guides

$$P_f^{\text{tree}}(t) = T_f^{(3D)}\big((\pi - 0.1)\eta(t)\big)(X_f), \qquad P_f^{\text{exact}}(t) = T_f^{(3D)}\big(\pi\eta(t)\big)(X_f).$$

We also write

$$P^{\text{tree}}(t) = \big(P_f^{\text{tree}}(t)\big)_{f \in \mathcal{F}}, \qquad P^{\text{exact}}(t) = \big(P_f^{\text{exact}}(t)\big)_{f \in \mathcal{F}}$$

for the corresponding facewise families. The first is the capped tree pose used for the display-oriented legacy and balanced modes; the second is the full exact folded guide used by the seamless rigid mode.

Definition 14.1 (Rigid face fit)**.** *For a face f and a target point set*

$$Y_f = (y_{f,1}, \dots, y_{f,m_f}) \in (\mathbb{R}^3)^{m_f},$$

the implemented rigid fit is

$$\operatorname{RigidFit}(X_f, Y_f) = R_f X_f + u_f,$$

where $(R_f, u_f) \in SO(3) \times \mathbb{R}^3$ minimizes

$$\sum_{i=1}^{m_f} \|R_f x_{f,i} + u_f - y_{f,i}\|^2 .$$

Numerically this is the usual orthogonal Procrustes / Kabsch solve with the reflection branch removed.

Lemma 14.2 (Rigid fit preserves face geometry)**.** *For every f and every target point set Y_f, the point set*

$$\operatorname{RigidFit}(X_f, Y_f)$$

is congruent to X_f. In particular, for every pair of indices i, j,

$$\Big\| \big(\operatorname{RigidFit}(X_f, Y_f)\big)_i - \big(\operatorname{RigidFit}(X_f, Y_f)\big)_j \Big\| = \|x_{f,i} - x_{f,j}\| .$$

Proof. By definition,

$$\big(\mathrm{RigidFit}(X_f, Y_f)\big)_i = R_f x_{f,i} + u_f$$

with $R_f \in SO(3)$. Hence

$$\|(R_f x_{f,i} + u_f) - (R_f x_{f,j} + u_f)\| = \|R_f(x_{f,i} - x_{f,j})\| = \|x_{f,i} - x_{f,j}\|,$$

because orthogonal matrices preserve Euclidean norm. □

Legacy layered

Intuition. Legacy layered is the most conservative display mode. For most of the animation it simply shows the original tree-based rigid fold. Only near the end does it peel the faces apart into a visibly separated flat stack, so that the final layer order is easy to read even when many faces would otherwise collapse onto the same plane. The price is that the final peel-away is a display blend, not a physically rigid motion.

Definition 14.3 (Legacy layered stack)**.** *Let $m = |\mathcal{F}|$. Evaluate the nearly-flat exact pose at angle*

$$\theta_{\mathrm{nf}} = \pi - 0.08.$$

For each face f, let $(x_f^{\mathrm{nf}}, y_f^{\mathrm{nf}}, z_f^{\mathrm{nf}})$ be the centroid of

$$T_f^{(3D)}(\theta_{\mathrm{nf}})(X_f).$$

Order the faces by the lexicographic key

$$\big(z_f^{\mathrm{nf}}, y_f^{\mathrm{nf}}, x_f^{\mathrm{nf}}\big),$$

and let $r(f) \in \{0, \dots, m-1\}$ be the resulting rank. With

$$\mathrm{span}(X) = \big(\max_i X_i^{(1)} - \min_i X_i^{(1)}\big) + \big(\max_i X_i^{(2)} - \min_i X_i^{(2)}\big),$$

define the implemented legacy spacing

$$\Delta_{\mathrm{leg}} = \max\left\{\frac{\mathrm{span}(X)}{18m}, 0.02\right\}, \qquad h_f^{\mathrm{leg}} = \Delta_{\mathrm{leg}}\left(r(f) - \frac{m-1}{2}\right).$$

If $\Phi_f^{\mathrm{flat}} : \mathbb{R}^2 \to \mathbb{R}^2$ is the exact flat transform of face f, the legacy target stack is

$$L_f = \left\{\big(\Phi_f^{\mathrm{flat}}(\Pi(x_{f,i})), h_f^{\mathrm{leg}}\big)\right\}_{i=1}^{m_f}.$$

The rendered legacy motion is

$$P_f^{\mathrm{legacy}}(t) = \big(1 - s_{0.72,1}(t)\big) P_f^{\mathrm{tree}}(t) + s_{0.72,1}(t) L_f.$$

Proposition 14.4 (Legacy layered endpoints)**.** *For every face f,*

$$P_f^{\mathrm{legacy}}(0) = X_f, \qquad P_f^{\mathrm{legacy}}(t) = P_f^{\mathrm{tree}}(t) \text{ for } 0 \le t \le 0.72, \qquad P_f^{\mathrm{legacy}}(1) = L_f.$$

Moreover the path $t \mapsto P_f^{\mathrm{legacy}}(t)$ is continuous on $[0, 1]$.

Proof. Since $\eta(0) = 0$, one has

$$P_f^{\text{tree}}(0) = T_f^{(3D)}(0)(X_f) = X_f.$$

By definition of $s_{0.72,1}$, the blend factor vanishes on $[0, 0.72]$ and equals 1 at $t = 1$. Therefore the displayed endpoint identities follow immediately. Continuity follows from continuity of η, $s_{0.72,1}$, P_f^{tree}, and affine interpolation in Euclidean space. □

Proposition 14.5 (Legacy layered is generally not rigid during the settle window)**.** *Fix $t \in (0.72, 1)$ and abbreviate $s = s_{0.72,1}(t) \in (0, 1)$. For two vertices a, b of the same face, let*

$$\Delta_{ab}^{\text{tree}} = P_{f,a}^{\text{tree}}(t) - P_{f,b}^{\text{tree}}(t), \qquad \Delta_{ab}^{\text{flat}} = L_{f,a} - L_{f,b},$$

and let $\ell_{ab} = \|x_{f,a} - x_{f,b}\|$. Then

$$\left\| P_{f,a}^{\text{legacy}}(t) - P_{f,b}^{\text{legacy}}(t) \right\|^2 = \ell_{ab}^2 + 2s(1-s)\big(\Delta_{ab}^{\text{tree}} \cdot \Delta_{ab}^{\text{flat}} - \ell_{ab}^2\big).$$

In particular, unless the endpoint edge vectors coincide for every edge of the face, the late legacy blend is not a rigid motion.

Proof. Both $P_f^{\text{tree}}(t)$ and L_f are rigid images of X_f, hence

$$\left\|\Delta_{ab}^{\text{tree}}\right\| = \left\|\Delta_{ab}^{\text{flat}}\right\| = \ell_{ab}.$$

The blended edge vector is

$$(1-s)\Delta_{ab}^{\text{tree}} + s\Delta_{ab}^{\text{flat}},$$

so expanding its squared norm gives

$$\begin{aligned}\left\|(1-s)\Delta_{ab}^{\text{tree}} + s\Delta_{ab}^{\text{flat}}\right\|^2 &= (1-s)^2\ell_{ab}^2 + s^2\ell_{ab}^2 + 2s(1-s)\Delta_{ab}^{\text{tree}} \cdot \Delta_{ab}^{\text{flat}} \\ &= \ell_{ab}^2 + 2s(1-s)\big(\Delta_{ab}^{\text{tree}} \cdot \Delta_{ab}^{\text{flat}} - \ell_{ab}^2\big).\end{aligned}$$

This equals ℓ_{ab}^2 for all $s \in (0, 1)$ only in the exceptional case that every edge vector agrees between the two endpoint placements. □

Balanced stack

Intuition. Balanced stack starts from the same tree pose, but it tries to heal visible cracks before the final display blend. The program repeatedly averages the different copies of each shared vertex and then rigidly refits each whole face to those averaged targets. This does not attempt to recover the exact physical folded state. Its purpose is more modest and more visual: reduce the worst seam openings while preserving a small amount of display thickness in the final stack.

For a global vertex v, let

$$\mathcal{O}(v) = \{(f, i) : \text{the } i\text{th local vertex of face } f \text{ represents } v\}.$$

Given a facewise point collection $Q = (Q_f)_f$, define the averaged shared vertex location

$$a_v(Q) = \frac{1}{|\mathcal{O}(v)|} \sum_{(f,i) \in \mathcal{O}(v)} Q_{f,i}.$$

For $0 \le \lambda \le 1$, one relaxation sweep forms targets

$$\widetilde{Q}_{f,i} = (1-\lambda)Q_{f,i} + \lambda a_{v_{f,i}}(Q)$$

and then refits the whole face by

$$\big(\mathcal{U}_\lambda(Q)\big)_f = \mathrm{RigidFit}(X_f, \widetilde{Q}_f).$$

The k-sweep relaxation operator is

$$\mathcal{R}_{k,\lambda} = \mathcal{U}_\lambda^k.$$

Definition 14.6 (Balanced stack target and motion)**.** *Using the same rank order $r(f)$ as legacy layered, the thinner balanced stack uses spacing*

$$\Delta_{\mathrm{bal}} = \max\left\{\frac{\mathrm{span}(X)}{220m}, 0.006\right\}, \qquad h_f^{\mathrm{bal}} = \Delta_{\mathrm{bal}}\left(r(f) - \frac{m-1}{2}\right),$$

and flat target

$$S_f = \left\{\big(\Phi_f^{\mathrm{flat}}(\Pi(x_{f,i})), h_f^{\mathrm{bal}}\big)\right\}_{i=1}^{m_f}.$$

Let

$$C_f = \big(\mathcal{R}_{6,0.9}(P^{\mathrm{tree}}(1))\big)_f, \qquad B_f = 0.64\, C_f + 0.36\, S_f.$$

At runtime the balanced mode uses

$$Q_f^{\mathrm{bal}}(t) = \big(\mathcal{R}_{k(t),0.88}(P^{\mathrm{tree}}(t))\big)_f, \qquad k(t) = \begin{cases} 4, & t < 0.65, \\ 5, & t \ge 0.65, \end{cases}$$

followed by the final display blend

$$P_f^{\mathrm{bal}}(t) = \big(1 - s_{0.76,1}(t)\big)Q_f^{\mathrm{bal}}(t) + s_{0.76,1}(t)B_f.$$

Proposition 14.7 (Balanced relaxation preserves per-face rigidity)**.** *For every point collection Q, every face of*

$$\mathcal{R}_{k,\lambda}(Q)$$

is congruent to the source face X_f.

Proof. Each sweep $\mathcal{U}_\lambda$ replaces a face by

$$\mathrm{RigidFit}(X_f, \widetilde{Q}_f),$$

which is congruent to X_f by the rigid-fit lemma. Iterating the same fact k times proves the claim. □

Proposition 14.8 (Closed rigid configurations are fixed points of the relaxation)**.** *Suppose that for each face f there are $(R_f, u_f) \in SO(3) \times \mathbb{R}^3$ with*

$$Q_f = R_f X_f + u_f,$$

and suppose moreover that whenever two local vertices represent the same global vertex, their points in Q coincide exactly. Then for every $\lambda \in [0,1]$,

$$\mathcal{U}_\lambda(Q) = Q.$$

Proof. If all copies of a global vertex already coincide, then

$$a_v(Q) = Q_{f,i}$$

for every $(f, i) \in \mathcal{O}(v)$, so the intermediate targets satisfy

$$\widetilde{Q}_{f,i} = Q_{f,i}.$$

Thus $\mathcal{U}_\lambda$ rigidly fits X_f to the exact rigid image Q_f itself. Because every exact face has positive area, at least three noncollinear points determine the rigid transform uniquely, so the zero-residual fit returns Q_f unchanged. □

Corollary 14.9 (Balanced endpoints)**.** *For every face f,*

$$P_f^{\mathrm{bal}}(0) = X_f, \qquad P_f^{\mathrm{bal}}(1) = B_f.$$

Proof. At $t = 0$ one has $P_f^{\mathrm{tree}}(0) = X_f$, and all shared copies of each vertex coincide. The preceding proposition therefore gives

$$Q_f^{\mathrm{bal}}(0) = X_f.$$

Also $s_{0.76,1}(0) = 0$ and $s_{0.76,1}(1) = 1$, so the displayed endpoint formula follows immediately from the final blend definition. □

Remark 14.10. As in legacy layered mode, the last affine blend toward B_f is generally not itself a rigid motion; the rigid guarantee applies to the relaxation output $Q_f^{\mathrm{bal}}(t)$ before the final display blend.

Seamless rigid

Intuition. Seamless rigid is the exact-only mode that tries to remove both visual defects at once: it wants connected faces in the middle of the motion, and it wants the last frame to be the true folded figure rather than a display stack. The exact fold is therefore used only as a guide. At discrete sample times the code re-solves a new family of rigid face transforms that stays close to the exact guide, keeps neighboring hinge rotations compatible with that guide, and explicitly penalizes seam openings. The displayed motion is then built only from those accepted rigid samples.

Definition 14.11 (Seamless rigid samples)**.** *Let r be the root face and let*

$$t_j = \frac{j}{12}, \qquad j = 0, \dots, 12.$$

For each sample time t_j, define the exact guide points

$$G_f(t_j) = P_f^{\mathrm{exact}}(t_j),$$

and let

$$\mathrm{RigidFit}(X_f, G_f(t_j)) = \widehat{R}_{f,j} X_f + \widehat{u}_{f,j}$$

be the facewise guide fit. The root face is fixed at

$$R_{r,j} = I, \qquad u_{r,j} = 0.$$

For each global vertex v, let (f_v, i_v) denote the first stored occurrence in $\mathcal{O}(v)$. The three residual families are

$$E_j^{\text{seam}} = \sum_v \sum_{(f,i)\in\mathcal{O}(v)\setminus\{(f_v,i_v)\}} \|(R_{f,j}x_{f,i} + u_{f,j}) - (R_{f_v,j}x_{f_v,i_v} + u_{f_v,j})\|^2,$$

$$E_j^{\text{guide}} = \sum_{f\neq r}\sum_i \|R_{f,j}x_{f,i} + u_{f,j} - G_{f,i}(t_j)\|^2,$$

$$E_j^{\text{hinge}} = \sum_{(f,g)\in H} \left\|\log\Big((R_{g,j}R_{f,j}^\top)(\widehat{R}_{g,j}\widehat{R}_{f,j}^\top)^\top\Big)\right\|^2,$$

where H is the set of fold-adjacent face pairs. The implemented objective is

$$\mathcal{E}_j = 130\,E_j^{\text{seam}} + 0.34\,E_j^{\text{guide}} + 0.75\,E_j^{\text{hinge}}. \tag{2}$$

If $p^\star$ denotes the parameter vector of the exact final fit $F_f^\star = P_f^{\text{exact}}(1)$ and $\widehat{p}_j$ the guide-fit parameter vector, then the optimizer is started from

$$p_j^{(0)} = \begin{cases} \widehat{p}_j, & t_j \le 0.84, \\ (1-t_j)\widehat{p}_j + t_j p^\star, & t_j > 0.84. \end{cases}$$

If an accepted sample has incident copies

$$(a_e^+, b_e^+), \qquad (a_e^-, b_e^-)$$

of a shared edge e, define the edge mismatch

$$\text{gap}_e = \min\Big\{\max\{\|a_e^+ - a_e^-\|, \|b_e^+ - b_e^-\|\}, \max\{\|a_e^+ - b_e^-\|, \|b_e^+ - a_e^-\|\}\Big\}.$$

The sample is accepted exactly when

$$\text{Gap}_j = \max_e \text{gap}_e \le 0.02.$$

Between two solved neighboring samples $t_a < t_b$, the rendered in-between rotation of each face is the geodesic / SLERP interpolation

$$R_f(t) = R_{f,a}\exp\Big(\alpha(t)\log\big(R_{f,a}^\top R_{f,b}\big)\Big), \qquad \alpha(t) = \frac{t-t_a}{t_b-t_a},$$

with linearly interpolated translation

$$u_f(t) = (1-\alpha(t))u_{f,a} + \alpha(t)u_{f,b}.$$

If one of the immediate neighboring samples failed, the renderer holds the nearest solved exact sample instead of dropping to the older gappy panel relaxation.

Proposition 14.12 (Accepted seamless-rigid samples are facewise rigid and seam-certified)**.** *If sample t_j is accepted, then every face of that sample is congruent to X_f, and its maximum shared-edge mismatch satisfies*

$$\text{Gap}_j \le 0.02.$$

Proof. Accepted samples are stored only in the form

$$R_{f,j}X_f + u_{f,j},$$

so every face is a rigid image of X_f. The bound on Gap_j is exactly the acceptance criterion used by the implementation. □

Proposition 14.13 (Every interpolated seamless-rigid frame is facewise rigid)**.** *Suppose $t \in (t_a, t_b)$ lies between two solved neighboring samples. Then for every face f, the rendered in-between points have the form*

$$R_f(t)X_f + u_f(t)$$

with $R_f(t) \in SO(3)$. Hence each face remains congruent to X_f throughout the interpolated interval.

Proof. The matrix

$$\exp\Big(\alpha(t)\log\big(R_{f,a}^{\top}R_{f,b}\big)\Big)$$

lies in $SO(3)$, so $R_f(t) \in SO(3)$. Therefore

$$R_f(t)X_f + u_f(t)$$

is a rigid image of X_f, and the rigid-fit lemma gives congruence of all facewise distances. □

Proposition 14.14 (Seamless rigid endpoints and exact-model fallback behavior)**.** *For the exact model,*

$$P_f^{\mathrm{rigid}}(0) = X_f, \qquad P_f^{\mathrm{rigid}}(1) = F_f^{\star} = P_f^{\mathrm{exact}}(1).$$

Moreover the implementation stores solved anchor samples at $t_0 = 0$ and $t_{12} = 1$ from initialization, so every progress value is rendered from either a solved exact sample or an interpolation between solved exact samples; it does not need to switch back to the older legacy panel relaxation.

Proof. The code explicitly stores the initial source configuration as solved sample t_0 and the exact folded fit as solved sample t_{12}. It also returns those two states directly at $t = 0$ and $t = 1$. Since these two anchors always exist, every intermediate progress value has at least one solved sample at or before it and at least one solved sample at or after it. Hence the exact renderer can always return either a solved exact sample or an interpolation between solved exact samples. □

Triangle-mesh fallback

Intuition. If exact face reconstruction fails, the program does not claim a certified rigid-face motion. Instead it triangulates the available regions and propagates rigid motions along a triangle tree. This keeps a useful picture on screen, but the resulting preview is explicitly warning-level and heuristic.

The mesh fallback triangulates each polygonal region into triangles

$$\Delta.$$

Let $\theta_{\mathrm{mesh}}^{\max}$ denote the mesh preview angle cap, equal to 76° in the flattening case and 88° otherwise. If a tree edge between triangles corresponds to a fold edge e with approximate normalized dihedral weight α_e, then the child transform is

$$S_{\Delta'}(t) = R_{(p,q)}\big(\theta_{\mathrm{mesh}}^{\max}\eta(t)\,\alpha_e\big)S_{\Delta}(t).$$

If the tree edge is merely a triangulation edge, the child simply inherits the parent transform. The legacy and balanced display variants are then applied to these triangle poses in exactly the same style as above.

Proposition 14.15 (Triangle rigidity in mesh fallback)**.** *Before the late display blend, every triangle in the mesh fallback remains rigid, and every child triangle shares its tree edge exactly with its parent.*

Proof. Each triangle transform is obtained from its parent either by applying a rigid rotation about the common edge or by copying the parent transform unchanged. Hence every triangle pose is a rigid image of its source triangle, and the common tree edge is preserved exactly by construction. □

Remark 14.16. The mesh fallback remains intentionally non-certifying: it does not prove exact global face reconstruction, exact cycle closure, or exact overlap order.

15 Status predicates

The four displayed statuses are designed to keep the local and global statements separate.

Definition 15.1 (Implemented status semantics)**.**

$$\mathrm{Status}_{\mathrm{local}}(\mathcal{C}) = \begin{cases} \mathrm{Pass}, & \mathrm{Local}(\mathcal{C};\varepsilon), \\ \mathrm{Fail}, & \textit{otherwise}, \end{cases}$$

$$\mathrm{Status}_{\mathrm{assign}}(\mathcal{C}) = \begin{cases} \mathrm{Fail}, & \exists v \in V_{\mathrm{int}} : |\mu_\tau(v)| \neq 2 \textit{ with complete local assignment data}, \\ \mathrm{NotRun}, & \forall e \in E : \tau(e) = -1, \\ \mathrm{Warn}, & \exists e \in E : \tau(e) = -1 \textit{ and some assignments exist}, \\ \mathrm{Pass}, & \mathrm{Assign}(\mathcal{C}), \end{cases}$$

$$\mathrm{Status}_{\mathrm{global}}(\mathcal{C}) = \begin{cases} \mathrm{Fail}, & \neg\,\mathrm{Noncross}(X), \\ \mathrm{NotRun}, & E = \varnothing, \\ \mathrm{Unknown}, & \mathrm{Status}_{\mathrm{assign}}(\mathcal{C}) = \mathrm{NotRun}, \\ \mathrm{Pass}, \mathrm{Warn}, \mathrm{Fail}, & \textit{exact-current-geometry solver output}, \end{cases}$$

$$\mathrm{Status}_{\mathrm{preview}}(\mathcal{C}) = \begin{cases} \mathrm{NotRun}, & \textit{no preview attempt admissible}, \\ \mathrm{Pass}, \mathrm{Warn}, \mathrm{Fail}, & \textit{preview solver output}. \end{cases}$$

The automation layer also uses two compound predicates.

Definition 15.2 (Workflow-level green predicates)**.** *The local optimization loop uses* LocalGreen *above, while the full automation predicate is*

$$\begin{aligned} \mathrm{AllGreen}(\mathcal{C}) \iff & \mathrm{Status}_{\mathrm{local}}(\mathcal{C}) = \mathrm{Pass} \wedge \mathrm{Status}_{\mathrm{assign}}(\mathcal{C}) = \mathrm{Pass} \\ & \wedge \mathrm{Status}_{\mathrm{global}}(\mathcal{C}) = \mathrm{Pass} \wedge \mathrm{Status}_{\mathrm{preview}}(\mathcal{C}) = \mathrm{Pass}. \end{aligned}$$

Remark 15.3. Thus the word "green" has two different implemented meanings: one local and geometric, used during optimization; and one global four-status conjunction, used by the full search workflows.

16 Scope and limitations

The project combines rigorous local constraints with a partly rigorous and partly heuristic global pipeline. The following distinctions are therefore essential.

- The shortest-path parity repair is an implemented heuristic, not an optimality theorem.
- The Bern–Hayes-style wedge reduction is a local sign-reduction device derived from [1], not a complete multi-vertex flat-foldability classifier.
- The exact face-reflection solver certifies the current embedding when it passes, but the mesh fallback is only a guarded visualization.
- Warning-level outputs indicate exactly where the code leaves the realm of strict certification.

Theorem 16.1 (Implemented scope)**.** *The current pipeline is not a complete decision procedure for arbitrary multi-vertex global flat-foldability.*

Proof. The claim follows from the coexistence of heuristic parity repair, local-only sign reduction, and explicit mesh/reference fallbacks in the preview stage. □

References

[1] Marshall Bern and Barry Hayes. *The Complexity of Flat Origami.* SODA, 1996.

[2] Thomas C. Hull. *On the Mathematics of Flat Origamis.* Congressus Numerantium, 1994.

[3] Zachary Abel, Jason Cantarella, Erik D. Demaine, David Eppstein, Thomas C. Hull, Jason S. Ku, Robert J. Lang, and Tomohiro Tachi. *Rigid Origami Vertices: Conditions and Forcing Sets.* Journal of Computational Geometry, 2017.

[4] Tomohiro Tachi. *Simulation of Rigid Origami.* In *Origami*^{4}, 2009.

[5] David Eppstein. *A Parameterized Algorithm for Flat Folding.* arXiv:2306.11939, 2023.

[6] Thomas C. Hull and Inna Zakharevich. *Flat Origami Is Turing Complete.* arXiv:2309.07932, version consulted in 2025.

[7] Andreas Walker and Tino Stanković. *Algorithmic Design of Origami Mechanisms and Tessellations.* Communications Materials, 2022.